FASE 6 — ANÁLISE, INTEGRAÇÃO E IMPACTO

Relatório técnico do projeto GameZone

Enterprise Challenge · Palo Alto Networks

Enzo Furlan — RM 255795
Gabriel Camara da Silva — RM 573006
Gustavo Aiello Ribeiro Amado — RM 568935
Guilherme Ribeiro Rosa — RM 572619

Outubro e novembro de 2026

Repositório: https://github.com/GameZone-source/GameZone-source.github.io
Site: https://gamezone-source.github.io/
Vídeo: LINK A INSERIR APÓS A PUBLICAÇÃO NÃO LISTADA

Sumário

Sumário.....
1. Visão geral do projeto.....
2. Parte 1 — Planejamento e organização da fase.....
  2.1 Escopo da fase.....
  2.2 Responsáveis pelas principais tarefas.....
  2.3 Dependências entre atividades.....
  2.4 Cronograma e acompanhamento (planejado × realizado).....
  2.5 Alterações relevantes no planejamento.....
3. Parte 2 — Testes e validação da solução.....
  3.1 Estratégia de teste.....
  3.2 Tabela de casos de teste.....
  3.3 Problemas encontrados, correções e re-teste.....
4. Parte 3 — Evolução tecnológica: Assistente GameZone.....
  4.1 Motivação e alinhamento com a proposta.....
  4.2 Arquitetura da solução.....
  4.3 Governança de IA.....
  4.4 Impacto digital.....
  4.5 Próximos passos.....
5. Conclusão.....
6. Vídeo de apresentação.....
Apêndice A — Como executar o projeto.....

1. Visão geral do projeto

O GameZone é uma plataforma de reviews próprias de jogos criada a partir do problema apresentado no Enterprise Challenge da Palo Alto Networks: a dificuldade de encontrar avaliações honestas e espaços confiáveis de discussão sobre games da nova geração. Até a Fase 5, o produto entregava o núcleo editorial: catálogo com busca e filtros, páginas detalhadas com critérios e pesos, favoritos persistidos no navegador, recomendações explicáveis, formulário de contato e endpoint público com as reviews.

A visão de longo prazo do grupo é evoluir o GameZone para um fórum e rede social focada em jogos. Na Fase 6, o trabalho avançou nessa direção com três frentes: a organização estruturada do trabalho da fase (Parte 1), a validação sistemática do que já estava construído (Parte 2) e uma evolução tecnológica alinhada à proposta — o Assistente GameZone, um agente de inteligência artificial que recomenda jogos a partir das reviews da equipe, sempre explicando o motivo de cada sugestão (Parte 3).

O site continua 100% estático e publicado no GitHub Pages, agora com a rota /assistente, o novo menu e uma suíte de testes ampliada: 55 testes de unidade e componente (Vitest) e 32 verificações automatizadas de acessibilidade (Playwright + axe), todas aprovadas.

2. Parte 1 — Planejamento e organização da fase

2.1 Escopo da fase

A partir do estado do projeto e da evolução prevista no README, o grupo definiu o seguinte escopo para a Fase 6:

- Evolução tecnológica: assistente de recomendação de jogos com IA local e explicável (Parte 3 da atividade).
- Refatoração: extração do motor de recomendação da Descoberta para uma biblioteca reutilizável (lib/recommendation.js), evitando duplicação entre a Descoberta e o assistente.
- Qualidade: bateria estruturada de testes das funcionalidades existentes e do novo assistente (Parte 2 da atividade), incluindo acessibilidade.
- Documentação: relatório, apresentação, vídeo e atualização do README.

2.2 Responsáveis pelas principais tarefas

Integrante	RM	Responsabilidades principais
Enzo Furlan	255795	Planejamento e acompanhamento; refatoração do motor de recomendação (lib/recommendation.js)
Gabriel Camara da	573006	Lógica do assistente: interpretação de pedidos e geração de

Integrante	RM	Responsabilidades principais
Silva		respostas (lib/assistant.js)
Gustavo Aiello Ribeiro Amado	568935	Interface do assistente, integração no site e acessibilidade (Chatbot.jsx, /assistente)
Guilherme Ribeiro Rosa	572619	Testes automatizados e guiados, correções e documentação da fase

2.3 Dependências entre atividades

As dependências foram mapeadas antes da execução para orientar a ordem de trabalho:

Atividade	Depende de	Responsável	Efeito
Extração do motor de recomendação (lib/recommendation.js)	—	Enzo Furlan	Bloqueia o assistente: ele reaproveita essas funções
Lógica do assistente (lib/assistant.js)	Motor de recomendação extraído	Gabriel Camara	Bloqueia a interface e os testes
Página /assistente + componente Chatbot	Lógica do assistente pronta	Gustavo Aiello	Bloqueia os testes de interface e a auditoria de acessibilidade
Testes automatizados (Vitest + Playwright)	Interface publicada na rota	Guilherme Ribeiro	Bloqueia a bateria final e o relatório
Bateria guiada de testes (CT-01 a CT-10)	App rodando em desenvolvimento	Guilherme Ribeiro	Correções reabrem o ciclo de teste
Relatório, slides e vídeo	Testes concluídos	Todos	Entrega final

2.4 Cronograma e acompanhamento (planejado × realizado)

Período	Datas	Atividades previstas	Situação
Semana 1	19/10 a 25/10	Planejamento da fase; levantamento dos casos de teste das funcionalidades existentes; refatoração do motor de recomendação em lib/recommendation.js	Concluído
Semana 2	26/10 a 01/11	Lógica do assistente (interpretação de pedidos e respostas); componente Chatbot e página /assistente; entrada no menu	Concluído
Semana 3	02/11 a 08/11	Testes automatizados do assistente; bateria guiada CT-01 a CT-10; correção dos problemas encontrados; auditoria de acessibilidade nas 8 rotas	Concluído
Semana 4	09/11 a 13/11	Relatório técnico, apresentação, gravação e publicação do vídeo, revisão final e envio do pacote na FIAP ON	Em andamento

O acompanhamento foi feito semanalmente pelo grupo, comparando o planejado com o realizado. Todas as atividades das semanas 1 a 3 foram concluídas dentro do prazo; a semana 4 está dedicada aos entregáveis finais.

2.5 Alterações relevantes no planejamento

Duas alterações foram registradas durante o desenvolvimento, ambas com motivo documentado:

- **Inclusão da tarefa "casamento tolerante de plataformas":** a bateria de testes (CT-08) revelou que o assistente negava plataformas compatíveis por comparação literal de strings ("Xbox" contra "Xbox Series"). A correção entrou no escopo da semana 3 como tarefa corretiva, com teste de regressão.
- **Antecipação da auditoria de acessibilidade:** o problema de foco inicial detectado em /assistente mostrou que auditar a rota nova junto com o desenvolvimento (e não apenas ao final) encurtava o ciclo de correção. A suíte passou a rodar a cada alteração relevante.

3. Parte 2 — Testes e validação da solução

3.1 Estratégia de teste

A validação combinou duas frentes complementares. A bateria guiada (CT-01 a CT-10) exercita cenários reais de uso no navegador, registrando entrada, resultado esperado e resultado obtido. A suíte automatizada cobre unidades puras (interpretação de pedidos, ranking de recomendações), componentes de interface (conversa do assistente, validação do formulário, navegação) e acessibilidade (axe WCAG 2.1/2.2 AA em todas as rotas, em visões desktop e mobile).

Resultado consolidado: 10/10 cenários guiados aprovados, 55 testes automatizados aprovados (Vitest) e 32 verificações de acessibilidade aprovadas (Playwright + axe).

3.2 Tabela de casos de teste

ID	Funcionalidade	Cenário	Entrada	Resultado esperado	Resultado obtido	Status
CT-01	Busca no catálogo	Buscar um jogo existente pelo nome	Digitar "elden" no campo Buscar jogo	Somente Elden Ring é listado e o contador de resultados é atualizado	1 resultado exibido; contador atualizado	Aprovado
CT-02	Filtros combinados	Filtrar por gênero Aventura e	Selecionar as duas opções nos	Somente No Man's Sky (Aventura, R$	1 resultado: No Man's Sky	Aprovado

ID	Funcionalidade	Cenário	Entrada	Resultado esperado	Resultado obtido	Status
		preço "Até R$199"	filtros	199) é listado		
CT-03	Ordenação	Ordenar o catálogo por maior nota	Selecionar "Maior nota" em Ordenar	Elden Ring (9.4) aparece primeiro	Elden Ring em primeiro lugar com nota 9.4	Aprovado
CT-04	Favoritos persistidos	Salvar um jogo e recarregar a página	Clicar em "Salvar" e recarregar / descoberta	Botão permanece "Salvo" após recarregar	Estado persistido via localStorage; 1 item "Salvo" após reload	Aprovado
CT-05	Validação do formulário de contato	Enviar o formulário vazio	Clicar em "Enviar sugestão" sem preencher	Mensagens de erro exibidas e foco no primeiro campo inválido (nome)	2 alertas exibidos; foco em #contato-nome	Aprovado
CT-06	Endpoint público de reviews	Consumir / api/reviews	GET na URL /api/reviews	HTTP 200 com as 6 reviews publicadas	HTTP 200; total = 6	Aprovado
CT-07	Assistente: pedido por gênero	Recomendação de RPG com justificativa	Enviar "recomende um RPG"	Sugere jogos de RPG com nota e motivo explicados	Elden Ring sugerido com "É do gênero RPG... Nota editorial 9.4..."	Aprovado
CT-08	Assistente: pedido composto	Gênero + plataforma + preço na mesma frase	Enviar "jogo de corrida para Xbox até 200"	Sugere Forza Horizon 5 e indica compatibilidade com Xbox	Forza Horizon 5 sugerido; motivo "Disponível para Xbox."	Aprovado (corrigido — 3.3)
CT-09	Assistente: sem correspondência	Pedido sem jogo compatível no catálogo	Enviar "estratégia até 10"	Resposta honesta informando que não há jogo abaixo de R$ 10	"Não achei reviews do catálogo abaixo de R$ 10... o mais próximo é Civilization VI, por R$ 129."	Aprovado
CT-10	Assistente: fora de escopo	Pergunta sem relação com o catálogo	Enviar "quem ganhou a copa de 2002?"	Fallback honesto indicando os limites do assistente	"Não entendi esse pedido..." com sugestão de pedir ajuda	Aprovado
TA-01	Favoritos e recomendações (automatizado)	Ranking e justificativas do radar	14 asserções em tests/lib/recommendation.test.js	Ranking exclui favoritos, ordena por score e explica cada sugestão	Testes aprovados	Aprovado
TA-02	Assistente (automatiz	Interpretação e respostas	20 asserções em	Intenções classificadas corretamente;	Testes aprovados	Aprovado

ID	Funcionalidade	Cenário	Entrada	Resultado esperado	Resultado obtido	Status
	ado)	do agente	tests/lib/assistant.test.js	respostas sempre com justificativa ou fallback honesto		
TA-03	Interface do assistente (automatizado)	Fluxo de conversa no componente	4 testes em tests/components/Chatbot.test.jsx	Resposta aparece na conversa; favoritos ficam fora das sugestões	Testes aprovados	Aprovado
TA-04	Acessibilidade (automatizado)	WCAG 2.1/2.2 AA nas 8 rotas do site	Playwright + axe (desktop e mobile)	Nenhuma violação séria ou crítica; skip link funcional em todas as rotas	32 verificações aprovadas	Aprovado (correção — 3.3)

3.3 Problemas encontrados, correções e re-teste

Problema	Descrição	Ajuste realizado	Re-teste
Assistente roubava o foco inicial da página	Na rota /assistente, o teste de skip link falhou: ao pressionar Tab, o foco ia para a conversa em vez do link "Pular para o conteúdo principal". O componente focava o registro da conversa na montagem inicial, porque a primeira mensagem já é do assistente.	O foco automático passou a acontecer somente após a primeira interação do usuário (referência userInteractedRef).	Re-execução da suíte: 32/32 verificações aprovadas, incluindo /assistente em desktop e mobile.
Casamento literal de plataforma gerava negativa indevida	No cenário CT-08, o assistente respondia "não encontramos Xbox" para Forza Horizon 5, embora a review liste "Xbox Series" e "Xbox One" — o casamento exato de strings não reconhecia a família de plataformas.	Criada a função platformMatches, que compara os nomes normalizados de forma tolerante ("xbox" casa com "xbox series"); teste de regressão adicionado.	CT-08 re-executado: resposta correta "Disponível para Xbox."; suíte completa segue aprovada (55 testes).

Nos dois casos, o problema foi documentado, corrigido e o teste foi executado novamente, com aprovação. Os testes de regressão criados nas correções passaram a fazer parte da suíte permanente.

4. Parte 3 — Evolução tecnológica: Assistente GameZone

4.1 Motivação e alinhamento com a proposta

A evolução escolhida foi a criação de um agente de inteligência artificial: o Assistente GameZone, disponível na rota /assistente. A escolha faz sentido para a proposta por três motivos. Primeiro, o chatbot já constava como evolução planejada no README do projeto, ou seja, a fase concretizou um passo do roadmap em vez de criar um desvio. Segundo, recomendação é o coração do produto: a plataforma nasceu para ajudar o jogador a decidir o que jogar, e o assistente reduz a fricção entre "ter vontade de jogar algo" e "encontrar o jogo certo". Terceiro, prepara o terreno para a futura comunidade (fórum e rede social), na qual atendimento e moderação assistida por IA serão necessários.

4.2 Arquitetura da solução

O site é exportado estaticamente (output: export no Next.js) e publicado no GitHub Pages, sem servidor próprio. Por isso, o assistente foi construído como um agente local, determinístico e auditável, dividido em três camadas:

- **Interpretação (lib/assistant.js):** normaliza o texto do usuário (minúsculas, sem acentos) e classifica a intenção em saudação, ajuda, consulta a um jogo ou recomendação, extraindo gênero, plataforma e faixa de preço a partir de dicionários.
- **Raciocínio (lib/recommendation.js):** motor de recomendação extraído da Descoberta nesta fase; pontua cada review pela nota editorial e pela afinidade com os favoritos do jogador, gerando a justificativa de cada sugestão.
- **Conversa (components/Chatbot.jsx):** interface de chat acessível (role="log" com aria-live), integrada aos favoritos salvos no navegador.

A resposta nunca é uma caixa-preta: cada jogo sugerido vem acompanhado do motivo (gênero pedido, nota editorial, compatibilidade de plataforma, custo-benefício) e do limite do agente declarado na própria interface.

Trecho — interpretação do pedido

```
// Interpreta a mensagem e devolve a intenção estruturada.
// Tipos: greeting | help | detail | recommendation | unknown
export function parseAssistantIntent(message, reviews = []) {
  const normalized = normalizeText(message)

  if (/^(oi|ola|bom dia|boa tarde|boa noite)\\b/.test(normalized)) return { type: 'greeting' }
  if (/(ajuda|como funciona|o que voce pode)/.test(normalized)) return { type: 'help' }

  const genres = matchDictionary(normalized, GENRE_KEYWORDS)
  const platforms = matchDictionary(normalized, PLATFORM_KEYWORDS)
  const maxPrice = extractMaxPrice(normalized)
  const detail = findReviewByTitle(normalized, reviews)
```

```
  if (genres.length || platforms.length || maxPrice !== null) {
    return { type: 'recommendation', genres, platforms, maxPrice }
  }
  if (detail) return { type: 'detail', review: detail }
  return { type: 'unknown' }
}
```

Trecho — resposta ranqueada e explicável

```
const ranked = pool
  .map((review) => buildRecommendation(review, intent.genres || [], intent.platforms || [],
genreWeight))
  .sort((a, b) => b.score - a.score)
  .slice(0, 3)


// buildRecommendation monta a justificativa de cada sugestão:
reasons.push(`É do gênero ${review.genre}, que você pediu.`)
reasons.push(`Nota editorial ${review.score.toFixed(1)}, entre as melhores do catálogo.`)
reasons.push(`Disponível para ${matches.join(' e ')}.`)
```

4.3 Governança de IA

Em linha com os critérios do desafio (segurança e uso responsável), o assistente adota salvaguardas concretas:

- Explicabilidade: toda sugestão exibe as razões que a produziram; nenhuma decisão é opaca.
- Limites declarados: a interface informa que o agente só conhece as reviews publicadas pela equipe e que não decide a compra pelo usuário.
- Fallback honesto: quando não entende o pedido ou não há jogo compatível, o assistente diz isso com clareza (CT-09 e CT-10) em vez de inventar recomendações.
- Base de dados controlada: o agente responde apenas a partir do catálogo editorial próprio, sem consumir conteúdo externo não verificado.
- Auditoria: o comportamento é coberto por testes automatizados, e o código é público no repositório do grupo.

4.4 Impacto digital

Para o usuário, o assistente transforma um catálogo estático em uma conversa: em uma frase, o jogador expressa gênero, plataforma e orçamento e recebe até três sugestões justificadas, respeitando o que já salvou no radar. Para o produto, cria o hábito de retorno ao site e estabelece o padrão de "IA explicável" que o grupo pretende manter nas próximas fases — inclusive na

moderação da futura comunidade, em que cada ação automatizada deverá ser justificada ao usuário.

4.5 Próximos passos

- Conectar o agente a um modelo de linguagem quando o projeto ganhar backend, mantendo a camada de justificativa e os limites declarados.
- Fórum com contas de usuário e moderação assistida por IA, reaproveitando o padrão de explicabilidade.
- Personalização contínua: aprender com os jogos abertos e pesquisados, mantendo o motivo visível em cada sugestão.

5. Conclusão

A Fase 6 cumpriu os três objetivos da atividade. O trabalho foi planejado com responsáveis, dependências e cronograma — e o realizado acompanhou o planejado, com duas alterações pontuais documentadas. A solução foi validada de forma estruturada: 10 cenários guiados, 55 testes automatizados e 32 verificações de acessibilidade aprovados, com dois problemas encontrados, corrigidos e re-testados. Por fim, a evolução tecnológica escolhida (agente de IA de recomendação) avança o roadmap do produto com uma solução explicável, auditável e coerente com a meta de transformar o GameZone em uma comunidade saudável de jogadores.

6. Vídeo de apresentação

O vídeo de até 5 minutos será publicado no YouTube de forma não listada e o link será inserido abaixo e na apresentação. Roteiro planejado:

Tempo	Bloco	Conteúdo
0:00 a 0:40	Abertura e contexto	Apresentar o GameZone, o problema do Enterprise Challenge e o que já existia até a Fase 5 (catálogo, busca, favoritos, recomendações explicáveis).
0:40 a 1:30	Parte 1 — Planejamento	Mostrar o cronograma da fase, a divisão de tarefas entre os integrantes e como o planejado foi acompanhado contra o realizado.
1:30 a 2:40	Parte 2 — Testes	Percorrer a tabela de testes: destacar a bateria guiada (CT-01 a CT-10), os 55 testes automatizados e os dois problemas encontrados e corrigidos (foco inicial e casamento de plataforma).
2:40 a 4:20	Parte 3 — Evolução: assistente de IA	Demonstração ao vivo em /assistente: pedido por gênero, pedido composto (gênero + plataforma + preço), consulta a um jogo específico e o fallback honesto. Explicar a arquitetura local e a governança (explicabilidade e limites declarados).
4:20 a	Impacto e	Impacto para o usuário, próximos passos (fórum e

Tempo	Bloco	Conteúdo
5:00	encerramento	moderação assistida) e convite para acessar o repositório e o site publicados.

Vídeo: LINK A INSERIR APÓS A PUBLICAÇÃO NÃO LISTADA

Apêndice A — Como executar o projeto

- Pré-requisitos: Node.js 20 ou superior e npm.
- Instalação: git clone https://github.com/GameZone-source/GameZone-source.github.io.git && cd GameZone-source.github.io && npm install
- Ambiente de desenvolvimento: npm run dev (http://localhost:3000).
- Testes automatizados: npm test (Vitest) e npm run test:a11y (Playwright, com servidor de desenvolvimento).
- Build de produção: npm run build (gera a pasta out/ para publicação estática).

A rota nova da fase é /assistente. O endpoint público de reviews continua disponível em /api/reviews.